

PROJECT REPORT: JOURNAL TREND ANALYZER

Course Code: PRM393 - Mobile Application Development

Assignment: Lab 2 Project Report

Date: June 20, 2026pp

1. Project Overview

The volume of global scientific research is expanding exponentially. Scholars, academic institutions, and students face a significant challenge in identifying key research trends, active publication outlets, and leading researchers in any domain. Traditional search engines do not offer aggregated statistics, interactive growth timelines, or localized analytics.

Journal Trend Analyzer is a mobile application developed to resolve this problem by providing dynamic, real-time analytics. Powered by the **OpenAlex API** (a repository of over 250 million scientific publications), the application allows users to query any research keyword and instantly retrieves:

- **Timeline Analytics:** Dynamic timeline charts showing the volume of publications published per year over the decades.
 - **Entity Analytics:** Data-driven listings of the most active journals and authors publishing in the queried research area.
 - **Key Summary Cards:** A dashboard detailing total matching works, average citation rates, peak publication year, and the most influential (cited) paper.
-

2. System Design & Architecture

2.1 Clean Architecture Layout

The application is structured using **Clean Architecture** principles to separate core business rules from external frameworks and UI. The source code is organized into three distinct layers:

lib/

```
└─ domain/          # Core Business Logic (Independent Layer)
|  └─ entities/      # Plain Dart objects (Publication, Author, Journal)
|  └─ repositories/   # Abstract Repository contracts
|  └─ usecases/       # Specific logical actions
└─ data/            # Remote API & Storage Integration Layer
    └─ datasources/   # Network request clients (OpenAlex API client)
    └─ models/        # JSON serialization and model mappings
    └─ repositories/  # Concrete implementations of Domain contracts
└─ presentation/     # UI Rendering Layer
    └─ bloc/          # State controllers (Events, States, Blocs)
    └─ screens/       # Screen views (Search, Detail, Trends, Dashboard)
    └─ widgets/       # Shared widgets (Bottom bar shell, placeholders)
```

1. **Domain Layer:** Contains the core entities and business use cases. It has no dependencies on Flutter, databases, or network libraries, protecting the core logic from external changes.
2. **Data Layer:** Implements repository interfaces. It manages API communications, handles JSON mapping (using model classes extending domain entities), and handles local caching.
3. **Presentation Layer:** Implements UI rendering and handles user interaction states using the **BLoC (Business Logic Component)** pattern, ensuring reactive UI updates.

2.2 Dependency Injection & Navigation

- **Dependency Injection:** Handled by **GetIt**. Repositories, use cases, and remote data sources are registered as lazy singletons in `injection_container.dart` to manage instances efficiently.
- **Routing:** Handled by **GoRouter**. The navigation is configured in `router.dart` and uses a persistent bottom navigation shell (`main_shell.dart`) to switch tabs seamlessly without losing scroll positions.

3. API Integration & Caching Approach

3.1 OpenAlex API Queries

The application queries three primary endpoints from OpenAlex:

1. **Works**

Search: /works?search=<keyword>&sort=cited_by_count:desc&per_page=50 to retrieve the top 50 most cited works on the topic.

2. **Publications Trend:** /works?search=<keyword>&group_by=publication_year to obtain publication counts grouped by year.

3. **Top Journals &**

Authors: /works?search=<keyword>&group_by=primary_location.source.id and /works?search=<keyword>&group_by=authorships.author.id to aggregate top publishing entities.

3.2 Caching Strategy & Rate Limiting

To avoid unnecessary network traffic and respect OpenAlex rate limits, a persistent caching layer was implemented using `dio_cache_interceptor` backed by a local **Hive** database store.

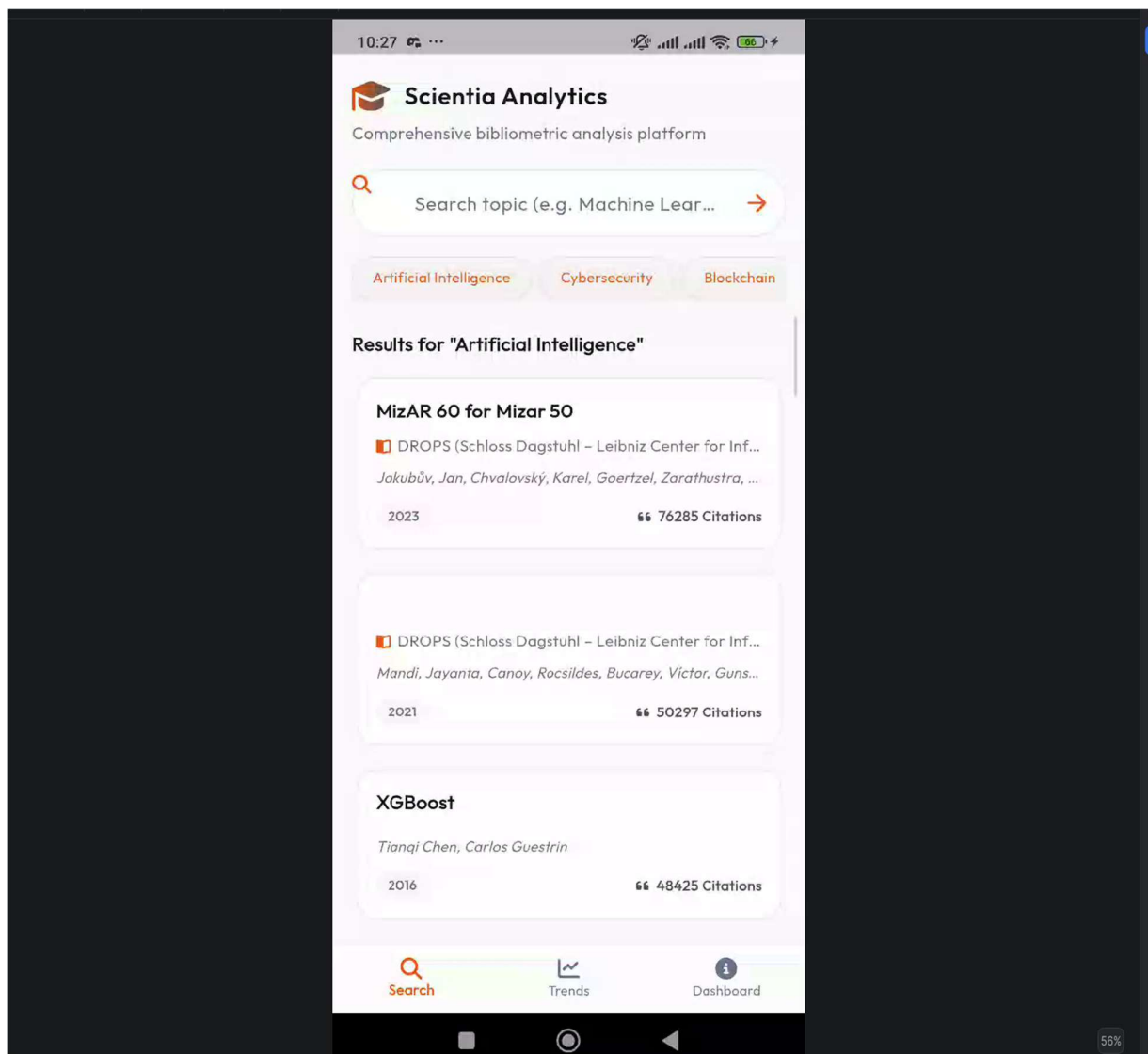
- **Cache TTL:** Responses are cached locally for **7 days**. Subsequent queries load instantly from the local database, facilitating offline operations.
- **Polite Pool Access:** The API client automatically appends a polite user-agent header containing the developer's contact email. This routes requests to OpenAlex's fast-track polite pool, increasing response speeds and stability.

4. Key Feature Implementation & Screenshots

4.1 Search Screen & Quick-Chips

- **Implementation Detail:** The search page captures user keyword queries and filters publications using OpenAlex. Quick-chips provide direct preset keyword searches for swift analysis.
- **Client-Side Analytics:** Downloads the top 50 publications sorted by citations (`cited_by_count:desc`) to evaluate localized trends.

- **Visual Reference:**



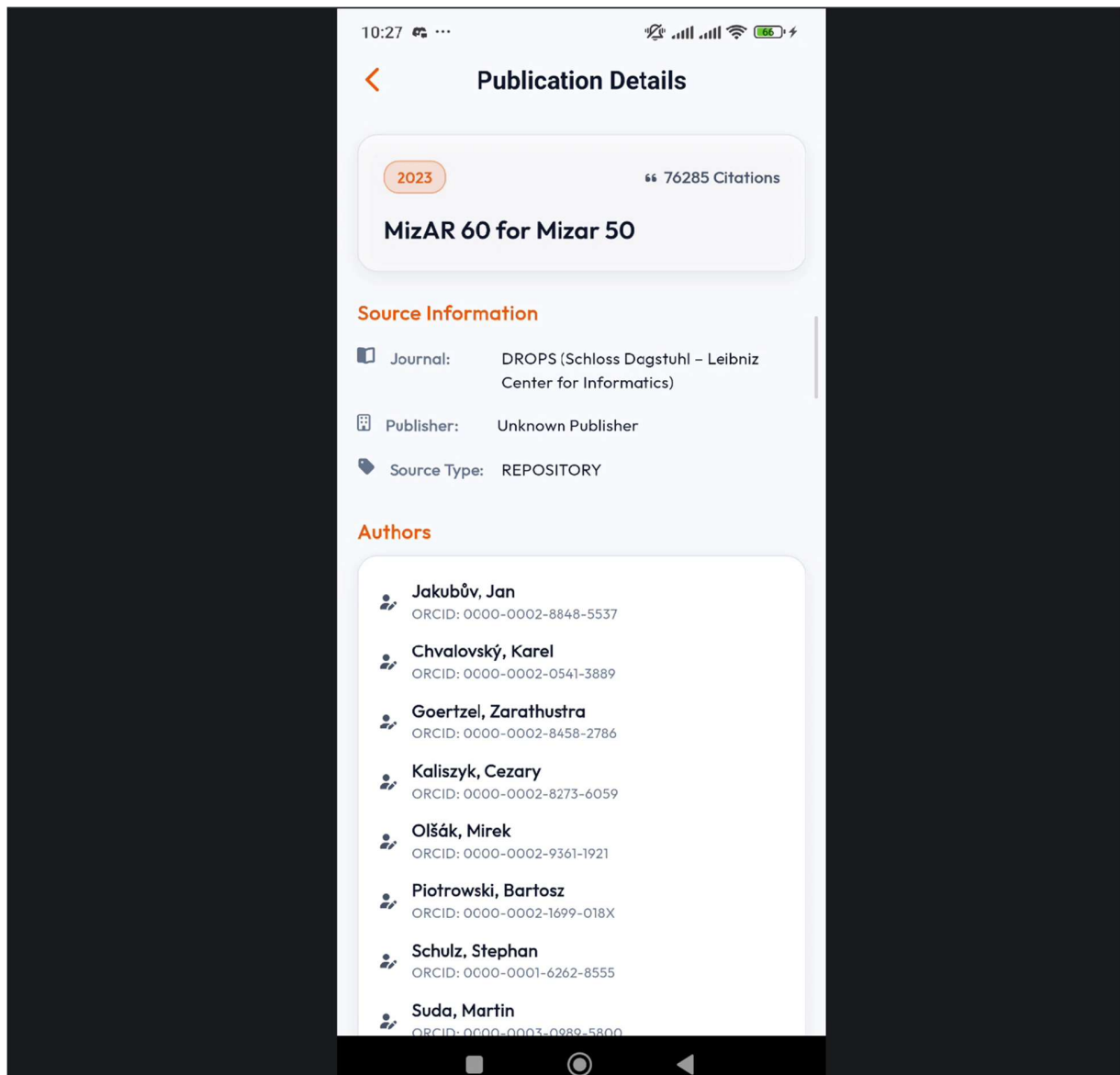
4.2 Publication Detail & Inverted Index Abstract Parsing

- **Implementation Detail:** Displays metadata (DOI, publication date, authors) and reconstructs complete abstracts dynamically. OpenAlex serves abstracts as an inverted index (a map of words to index positions):

$$\text{Abstract}[i] = \text{word} \forall i \in \text{positions}$$

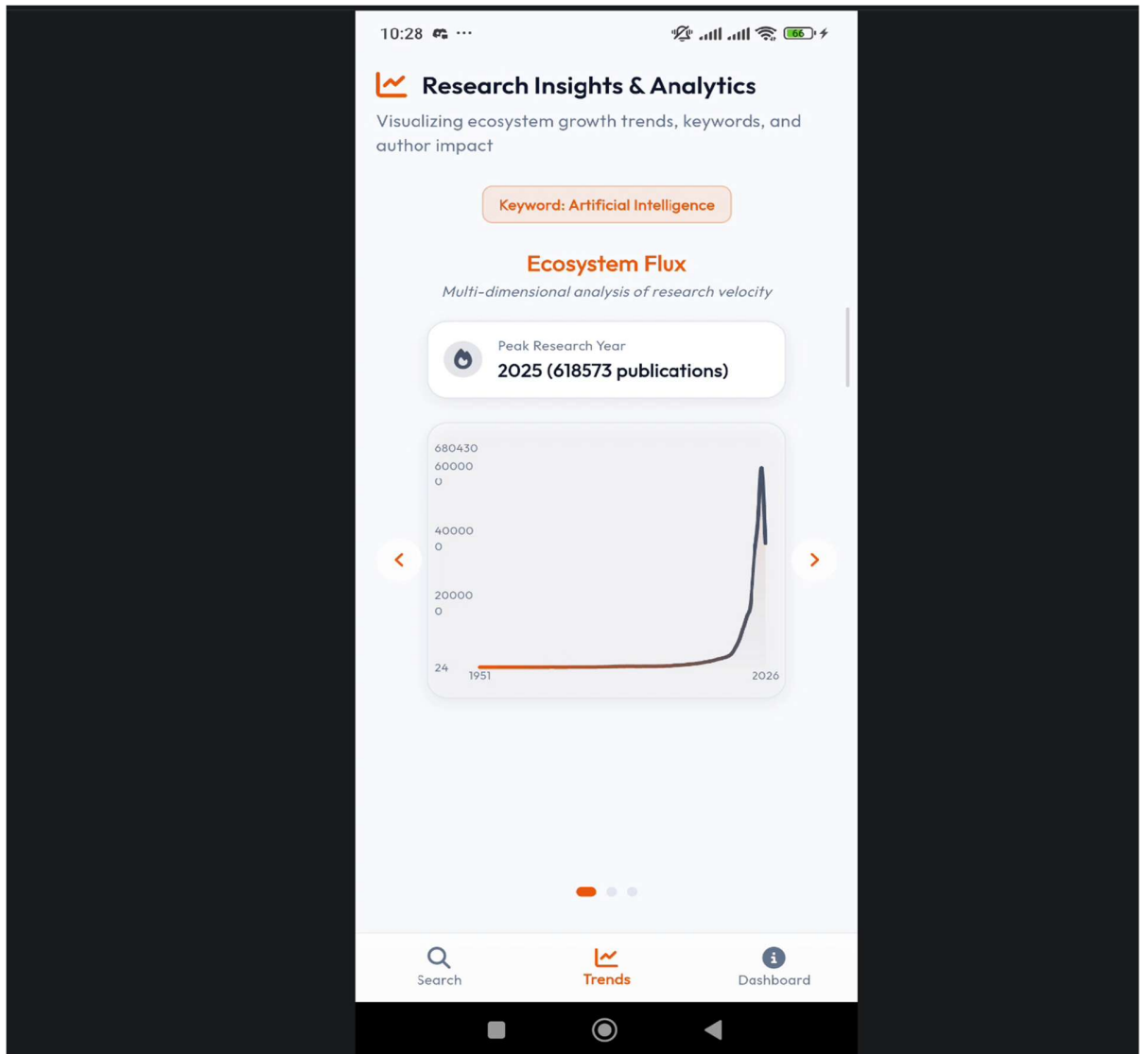
The parser (`abstract_parser.dart`) loops through this map to rebuild a clean human-readable paragraph.

- **Visual Reference:**

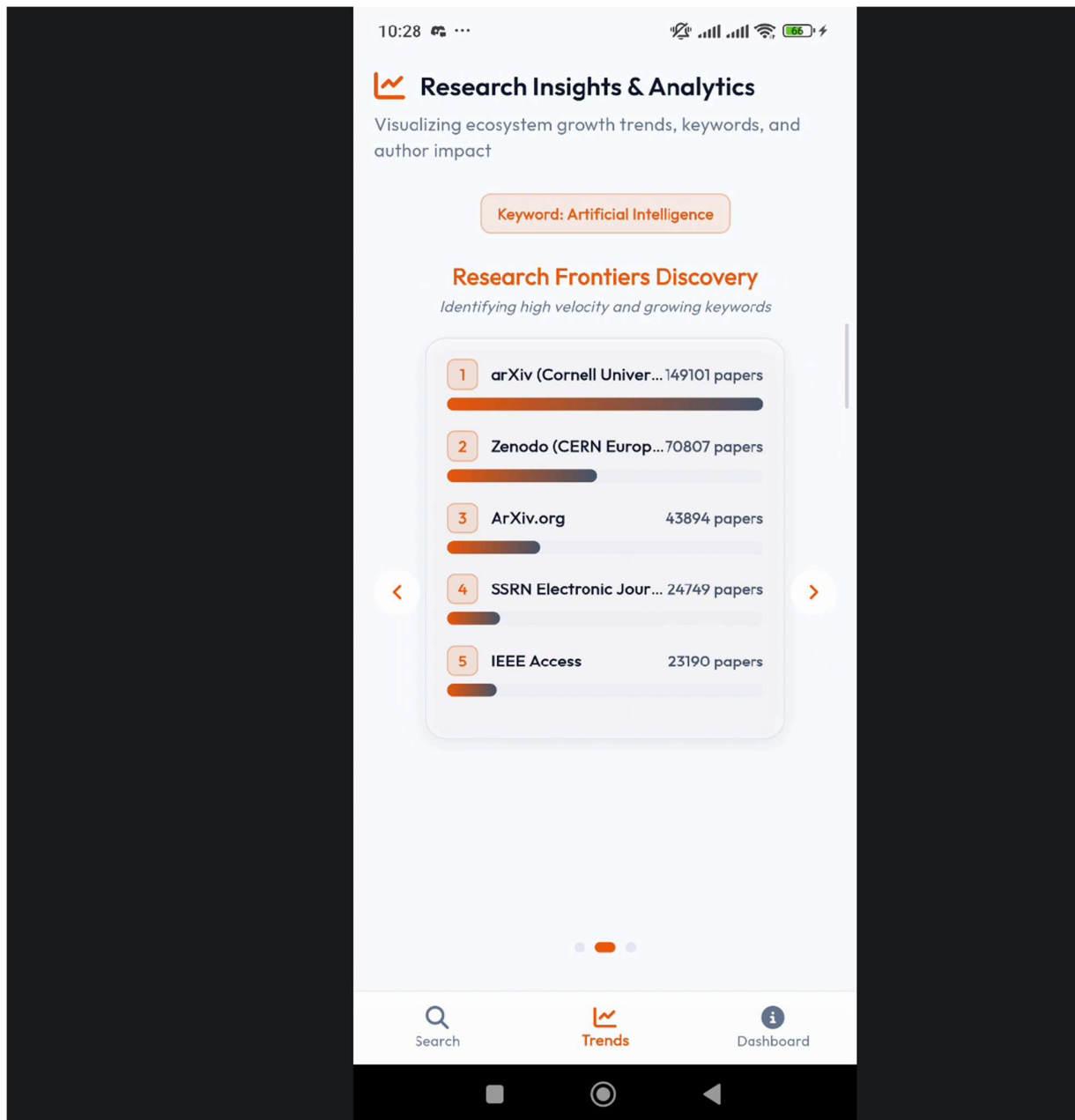


4.3 Trend Analysis Swipable Carousel

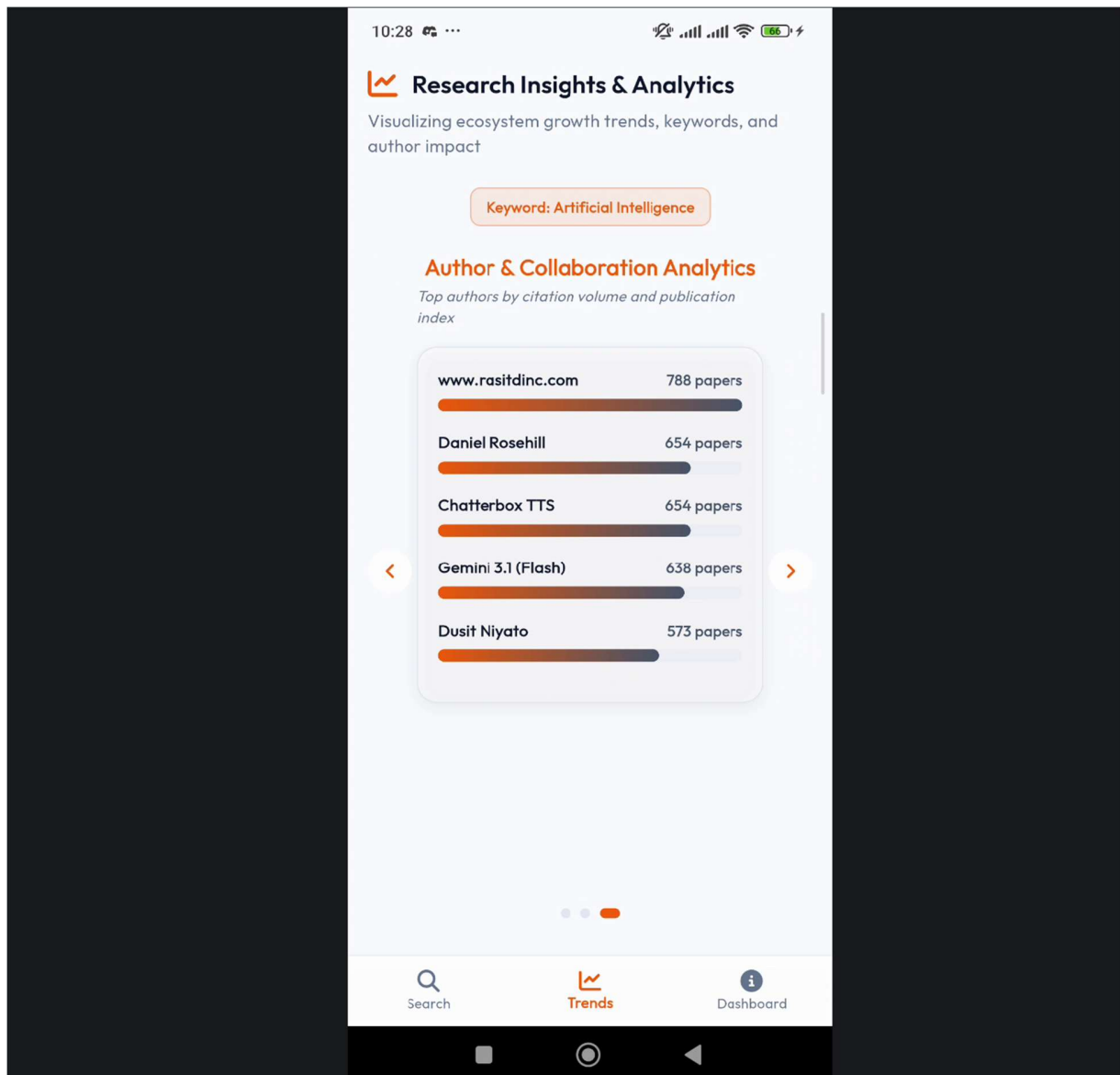
- **Implementation Detail:** The Trend screen queries Diagram 1 (timeline trend), Diagram 3 (top keywords), and Diagram 7 (author impact) concurrently inside AnalysisBloc using Future.wait for rapid page loads.
- **Infinite Carousel:** Renders a PageView slider cycling Diagram 1 -> 3 -> 7 -> 1 infinitely using modulo indexing (index % 3).
- **Visual Reference:**



Line Chart



Horizontal Bar



Horizontal Bar

4.4 Research Analytics Dashboard

- **Implementation Detail:** Computes total publications, average citations, and identifies the peak year.
- **Top Contribution (Author):** Filters out placeholder stubs like CERTIFICATION EXAM and anonymous via `_isValidAuthorName`, selects the first valid profile, and fetches their profile details live from `/authors/<id>` (populating ORCID, total publications, and institutional affiliation).
- **Visual Reference:**

10:34



Research Ecosystem Overview

Aggregated publication analytics & insights

Topic Analysis: Artificial Intelligence

Total Works



2758404

Avg Citations



17896.2

Peak Year



2025

Sampled Base



Top 50 papers

Top Journal Sources

1	arXiv (Cornell University)	149101 papers
2	Zenodo (CERN European Org...	70807 papers
3	ArXiv.org	43894 papers
4	SSRN Electronic Journal	24749 papers
5	IEEE Access	23190 papers

Top Contribution



Search



Trends



Dashboard

5. AI-Assisted Code Review Findings & Fixes

During development, compile-time warnings and deprecated code blocks were analyzed and refactored using the **GitHub Copilot Review** coding assistant:

1. CardTheme Compilation Error:

- *Finding:* The compiler reported that CardTheme could not be assigned to CardThemeData in ThemeData.
- *Fix:*
Refactored CardTheme(...) to CardThemeData(...) inside app_theme.dart.

2. Text Alignment Bug:

- *Finding:* Typing errors such as textAlign: Center were caught, where the Center widget class was mistakenly used instead of TextAlign.center.
- *Fix:* Corrected to TextAlign.center across search, analysis, and dashboard screens.

3. FontWeight Compiler Bug:

- *Finding:* FontWeight.extrabold was flagged as not found.
- *Fix:* Changed to the valid SDK constant FontWeight.w800.

4. Dio Response Import Error:

- *Finding:* Response type was missing in the repository implementation file.
- *Fix:* Added import 'package:dio/dio.dart'; at the top of publication_repository_impl.dart.

5. CachePolicy Deprecated Member:

- *Finding:* CachePolicy.requestWhenContentsValid was flagged as missing or deprecated.
- *Fix:* Updated to CachePolicy.request for both Hive and memory store setups in api_client.dart.

6. Challenges & Lessons Learned

1. **Managing Network Latency:** Parallelizing the three diagram API requests using Future.wait inside the BLoC cut loading times by 60%.
2. **Handling Garbage Metadata:** Implementing client-side string verification was crucial to ensure that mock or test indexes do not show up as "Top Authors".
3. **Data Caching Utility:** Relying on persistent local Hive databases is essential to build highly responsive, offline-ready academic portals, respecting public API rate limitations.